

چرخه جدید تولید نرم افزار با کدنویسی شهودی

از درخواست های نامنظم تا مهندسی عامل محور

مطالعه پیشنهادی: ۴۵ دقیقه وابستگی: پیش نیاز چهار جلسه بعدی

فهرست مطالب

- مقدمه: تغییری که از جنس ابزار نیست
- عامل چیست؟ مروری کوتاه اما دقیق
- طیف: از کدنویسی شهودی تا مهندسی عامل محور
- مهندسی زمینه: مهارت واقعی
- چرخه جدید تولید نرم افزار
- مهندسی مهار: آنچه مدل را احاطه می کند
- نقش در حال تحول توسعه دهنده
- عامل های کدنویسی در عمل
- اقتصاد توسعه مبتنی بر هوش مصنوعی
- از کجا شروع کنیم
- جمع بندی و سه اصل ماندگار
- خودآزمایی

۱. مقدمه: تغییری که از جنس ابزار نیست

برای بیشتر تاریخ محاسبات، برنامه نویسی نوعی «ترجمه» بوده است: مسئله را به زبان انسانی بفهم، راه حل را به زبان انتزاعی طراحی کن، و بعد آن را در نحوی بنویس که ماشین بتواند اجرا کند. هر گام از این زنجیره اصطکاک می سازد. آنچه امروز رخ داده، فرو ریختن این اصطکاک است.

عمیق‌ترین تغییر در مهندسی نرم‌افزار، یک زبان، چارچوب یا سرویس ابری تازه نیست. گذار از نوشتن کد به بیان قصد است؛ و اعتماد به سامانه‌های هوشمند برای ترجمه آن قصد به نرم‌افزاری که کار می‌کند.

این جابه‌جایی یک‌شبه رخ نداد. با تکمیل خودکار شروع شد – پیش‌بینی ساده‌ی نشانه‌ها در ویرایشگر. سپس پیشنهادها درون خطی آمد که می‌توانستند یک تابع کامل را بنویسند. بعد رابط‌های گفت‌وگویی اجازه دادند ویژگی‌ها را به زبان طبیعی توصیف کنیم و پیاده‌سازی بگیریم. و اکنون عامل‌های کاملاً خودگردان مخزن را کپی می‌کنند، تغییرهای چندفایلی را برنامه‌ریزی و اجرا می‌کنند، در محیط ایزوله آزمون می‌گیرند و درخواست ادغام می‌سازند – بدون آنکه انسانی حتی یک خط تایپ کند.

اعداد وضعیت موجود (اوایل ۲۰۲۶)

۸۵٪ توسعه‌دهندگان حرفه‌ای مرتب از عامل‌های کدنویسی استفاده می‌کنند، ۵۱٪ روزانه، و برآورد می‌شود ۴۱٪ از کدهای تازه‌نوشته‌شده تولید هوش مصنوعی باشد.

اما این دگرگونی یکنواخت یا ساده نیست. طیفی وجود دارد که یک سر آن «کدنویسی شهودی» است – توسعه‌دهنده درخواستی می‌دهد و هرچه برگشت را می‌پذیرد – و سر دیگرش «مهندسی عامل‌محور»، جایی که هوش مصنوعی موتور نیرومند پیاده‌سازی است، اما درون سامانه‌ای از قیدها، آزمون‌ها و حلقه‌های بازخورد آگاهانه طراحی شده کار می‌کند و انسان نظارت بر معماری، درستی و کیفیت را نگه می‌دارد.

چرا این تمایز، تمایزی مدیریتی هم هست

اگر به مدیر ارشد فناوری بگویید تیم شما سامانه‌ی پردازش پرداخت را «شهودی» کدنویسی می‌کند، باید زنگ خطر به صدا دربیاید – و درست هم هست. اگر به همان فرد بگویید تیم «مهندسی عامل‌محور» می‌کند و هوش مصنوعی زیر قیدهای انسانی طراحی شده پیاده‌سازی می‌کند و پوشش آزمون درستی را تضمین می‌کند، گفت‌وگو بنیادین دیگری است.

۲. عامل چیست؟ مروری کوتاه اما دقیق

پیش از هر چیز به تصویری مشترک نیاز داریم. **عامل هوش مصنوعی** سامانه‌ای نرم‌افزاری است که یک هدف را درک می‌کند، گام‌های رسیدن به آن را برنامه‌ریزی می‌کند، از راه ابزارها کنش انجام می‌دهد، نتیجه را مشاهده می‌کند و تکرار می‌کند تا به هدف برسد یا به شرط توقف بخورد.

تفاوت بنیادی با ربات گفت‌وگو در همین جاست: ربات گفت‌وگو پاسخی تولید می‌کند و منتظر درخواست بعدی می‌ماند؛ عامل حلقه‌ی خودش را می‌چرخاند. شما هدف را در بالا می‌گذارید و او در هر گام تصمیم می‌گیرد قدم بعدی

حلقهٔ عامل

ادراک ← **برنامه** ← **کنش** ← **مشاهده** ← **تکرار**. این حلقه، قلب تپندهٔ هر عامل است. هر چیزی که در ادامهٔ این دوره می‌بینید – از ابزارها و مهارت‌ها تا ارزیابی و امنیت – صورتی از همین حلقه یا مراقبتی بر آن است.

پنج جزء سازنده

جزء	نقش	اگر نباشد چه می‌شود
مدل	موتور استدلال؛ زمینهٔ جاری را می‌خواند و تصمیم می‌گیرد گام بعد چه باشد؛ اندیشهٔ بعدی، فراخوان ابزار بعدی، یا پیام بعدی.	سامانه فقط یک اسکریپت ثابت است.
ابزارها	اتصال مدل به جهان: API‌هایی که می‌تواند صدا بزند، کدی که اجرا می‌کند، پایگاه‌داده‌ای که پرس‌وجو می‌کند، عامل دیگری که کار را به او می‌سپارد.	عامل فقط حرف می‌زند، کاری نمی‌کند.
حافظه	وضعیت؛ یادآوری تعامل‌های گذشته، بازیابی قواعد پروژه، و نگه‌داشتن زمینه میان نشست‌ها تا هرگز از صفحهٔ سفید شروع نکند.	هر نشست از صفر شروع می‌شود و همان اشتباه‌ها تکرار می‌شوند.
ارکستراسیون	کدی که حلقه را می‌چرخاند: زمینهٔ هر فراخوان را می‌سازد، فراخوان ابزار را می‌فرستد، نتیجه را می‌گیرد و تصمیم می‌گیرد ادامه بدهد یا نه.	حلقه‌ای در کار نیست؛ فقط یک پرسش و پاسخ.
استقرار	آنچه نمونهٔ اولیه را به سرویس تبدیل می‌کند: میزبانی، هویت، رصدپذیری و زیرساخت تولید.	نمونهٔ اولیه روی لپ‌تاپ باقی می‌ماند.

۳. طیف: از کدنویسی شهودی تا مهندسی عامل محور

خاستگاه واژه

در فوریه ۲۰۲۵، آندری کارپش شیوه‌ای از برنامه‌نویسی را توصیف کرد که به سرعت در جامعه مهندسی نرم افزار طنین انداخت: «کاملاً خودت را به حس و حال بسپار، به نماها اعتماد کن، و فراموش کن که اصلاً کدی وجود دارد.» در این حالت، توسعه‌دهنده خواسته‌اش را به زبان طبیعی می‌گوید، خروجی را می‌پذیرد و وقتی چیزی می‌شکند، پیام خطا را در درخواست بعدی کپی می‌کند و می‌گوید درستش کن.

این اصطلاح فراگیر شد چون چیز واقعی‌ای را نام‌گذاری کرد: بسیاری از توسعه‌دهندگان از قبل همین‌طور کار می‌کردند اما واژه‌ای برایش نداشتند. اما ظرف چند ماه، «کدنویسی شهودی» به هر گردش کار کمک‌گرفته از هوش مصنوعی اطلاق شد و همین، معنایش را رقیق کرد. تا اوایل ۲۰۲۶ خود کارپش پذیرفت که قاب اولیه بیش از حد تنگ بوده و واژه مهندسی عامل محور را برای سر منضبط این طیف پیشنهاد کرد.

طیف، نه دوگانه

مفیدتر است این دو را نه به صورت یک انتخاب دوتایی، بلکه به صورت دو سر یک طیف ببینیم. تمایزدهنده این نیست که از هوش مصنوعی استفاده می‌کنید یا نه؛ این است که چه میزان ساختار، راستی‌آزمایی و قضاوت انسانی خروجی آن را احاطه کرده است.

بُعد	کدنویسی شهودی	کمک گرفتن ساختارمند	مهندسی عامل محور
بیان قصد	درخواست‌های گفتاری و سردستی	درخواست‌های دقیق همراه با مثال و قید	مشخصات رسمی، سند معماری، فایل‌های حافظه
راستی‌آزمایی	«به نظر کار می‌کند؟»	آزمون دستی، واریسی نمونه‌ای	مجموعه‌آزمون خودکار، دروازه‌های CI/CD، داوران مدل‌زبانی
فهم کدپایه	حداقلی؛ ممکن است کد تولیدشده اصلاً خوانده نشود	بازبینی گزینشی مسیرهای حساس	بازبینی جامع معماری؛ جزئیات پیاده‌سازی با عامل
مدیریت خطا	کپی پیام خطا و بازگرداندن به مدل	انسان علت ریشه‌ای را تشخیص می‌دهد، عامل اصلاح را پیاده می‌کند	عامل درون مرزهای تعریف‌شده خودتشخیص می‌دهد؛ مسائل معماری با انسان
دامنه مناسب	نمونه اولیه، اسکریپت، پروژه شخصی، هکاتون	ویژگی‌ها درون کدپایه‌های جاافتاده	سامانه‌های تولیدی، توسعه در مقیاس تیم
نمای ریسک	بالا؛ برای کد دورریختنی پذیرفتنی	متوسط؛ قضاوت انسانی در ایستگاه‌های کلیدی	پایین؛ راستی‌آزمایی نظام‌مند در هر مرحله

جدول ۱ – طیف از کدنویسی شهودی تا مهندسی عامل محور

نکته کاربردی

جای درست شما روی این طیف را **میزان مخاطره** تعیین می‌کند. یک نمونه اولیه آخر هفته می‌تواند کاملاً شهودی باشد. یک API تولیدی که تراکنش مالی را پردازش می‌کند، مهندسی عامل محور می‌طلبد. بیشتر کارهای واقعی جایی در میانه‌اند و مهارت، در دانستن این است که برای هر کار خط را کجا بکشید.

راستی‌آزمایی: تمایزدهنده اصلی

بزرگ‌ترین تفاوت دو سر طیف در نحوه راستی‌آزمایی خروجی‌هاست. در کدنویسی شهودی، راستی‌آزمایی اختیاری است؛ کد را اجرا می‌کنی و می‌بینی درست به نظر می‌رسد یا نه. در مهندسی عامل محور دو سازوکار با هم کار می‌کنند:

- **آزمون‌ها** بخش قطعی سامانه را می‌سنجند: تابعی با این ورودی باید آن خروجی را بدهد. داورشان کد است.

- **ارزیابی‌ها** بخش غیرقطعی را می‌سنجند: آیا عامل مسیر درستی از گام‌ها را طی کرد؟ ابزار درست را انتخاب کرد؟ پاسخ نهایی به حد کیفی رسید؟ داورشان مجموعه دادهٔ برچسب‌خورده، سنج‌های نمره‌دهی و داوران مدل‌زبانی‌اند.

قاعدهٔ سخت

بدون هر دو، کاری که انجام می‌دهید کدنویسی شهودی است – مهم نیست درخواست‌هایتان چقدر پیچیده و حرفه‌ای باشند.

۴. مهندسی زمینه: مهارت واقعی

با بلوغ این حوزه یک بینش کلیدی پدیدار شد: کیفیت کد تولیدشده کمتر به زیرکی درخواست‌ها و بیشتر به کیفیت زمینهای بستگی دارد که فراهم کرده‌اید. این همان چیزی است که **مهندسی زمینه** نام گرفته: تأمین اطلاعات غنی و ساختارمند دربارهٔ کدپایه، معماری، قراردادهای و قصد شما.

شش نوع زمینه

1. **دستورها:** نقش اصلی عامل، هدف‌ها و مرزهای عملیاتی‌اش.
2. **دانش:** سندهای بازتابی‌شده، نمودارهای معماری و دادهٔ تخصصی حوزه.
3. **حافظه:** گزارش‌های کوتاه‌مدت نشست (چه گذشت) و وضعیت ماندگار بلندمدت (این پروژه چیست).
4. **مثال‌ها:** نمونه‌های رفتاری چندشاتی و الگوهای مرجع درون کدپایه.
5. **ابزارها:** تعریف دقیق APIها، اسکریپت‌ها و سرویس‌های بیرونی قابل فراخوانی.
6. **حفاظ‌ها:** قیدهای سخت، قواعد قالب‌بندی و اعتبارسنجی‌های ایمنی.

زمینهٔ ایستا در برابر زمینهٔ پویا

زمینهٔ ایستا همیشه بارگذاری می‌شود: دستورهای سامانه، فایل‌های قاعده (`AGENTS.md` ، `CLAUDE.md` ، `GEMINI.md`)، حافظهٔ سراسری و تعریف شخصیت. این زمینه می‌گوید عامل کیست و چگونه رفتار می‌کند. گران است، چون هر توکنش در هر تعامل حاضر است، ربطی داشته باشد یا نه.

زمینهٔ پویا هنگام نیاز بارگذاری می‌شود: دستورهای مهارتی که با کار جاری هم‌خوان است، نتایج ابزارها، سندهای بازتابی‌شده و تاریخچهٔ پنجره‌ای نشست. کارآمد است، چون فقط وقتی هزینهٔ توکن می‌پردازید که اطلاعات لازم باشد.

این یک موازنه مهندسی واقعی است

زمینه ایستای بیش از حد، توکن هدر می دهد و سیگنال را رقیق می کند. زمینه ایستای کم از حد یعنی عامل قواعد حیاتی را فراموش می کند. بهترین سامانه ها این مرز را یک **تصمیم معماری درجه یک** می دانند: بازبینی شده و نسخه بندی شده، مثل هر پیکربندی دیگر.

مهارت های عامل: قوی ترین الگوی زمینه پویا

مهارت های عامل بسته های ساختارمند و قابل حملی از دانش رویه ای اند که عامل فقط وقتی بارشان می کند که کار اقتضا کند. به جای جاسازی هر دانش تخصصی در درخواست سامانه، عامل یک عمومی کار سبک می ماند که به اقتضا در نقش متخصص فرو می رود – سازوکاری به نام **افشای تدریجی**:

1. در آغاز، عامل فقط فراداده های سبک هر مهارت را می بیند.
2. وقتی کاری با یک مهارت تطبیق پیدا کرد، دستور کامل آن بارگذاری می شود.
3. مرجع تفصیلی فقط در صورت نیاز صریح خوانده می شود.

نتیجه این است که عامل می تواند ده ها توانایی تخصصی حمل کند و فقط برای همانی که همین حالا فعال است توکن بپردازد. مهارت ها به سرعت در عامل های کدنویسی و سکوی های سازمانی پذیرفته شدند، چون چهار مشکل دیرینه را حل می کنند: پوسیدگی زمینه بر اثر درخواست های انباشته، نبود حافظه رویه ای در مدل های زبانی، سربار عملیاتی معماری های چندعاملی، و نیاز به قابلیت حمل میان ابزارها و فروشندگان.

ادامه این بحث

جلسه سوم دوره کاملاً به همین موضوع اختصاص دارد: طراحی و مدیریت نشست، نوشتن و ارزیابی مهارت، ساخت حافظه ماندگار، و بهینه سازی اقتصاد توکن برای سامانه های تولیدی.

پرسش درست این نیست که «چطور هوش مصنوعی را فریب بدهم تا کد خوب بنویسد؟» پرسش درست این است: «یک هم تیمی تازه وارد چه چیزی باید بداند تا بتواند مؤثر مشارکت کند، و چطور آن دانش را به شکلی رمزگذاری کنم که هوش مصنوعی هم بتواند به کارش ببرد؟»

۵. چرخه جدید تولید نرم افزار

چرخه سنتی زیر فشار

چرخه تولید نرم افزار پیش تر هم یک دگرگونی بزرگ را از سر گذرانده است: گذار از فرایندهای آبشاری به مدل های تکرارشونده – دوره های چابک، یکپارچه سازی پیوسته، خطوط لوله DevOps و انتشار سریع. آن تغییر، حلقه های بازخورد را کوتاه کرد و استقرار را از رویدادی فصلی به فرایندی پیوسته بدل ساخت.

هوش مصنوعی این چرخه را به شدت فشرده می کند، اما **نامتوازن**: پیاده سازی ای که هفته ها می برد اکنون ساعت ها طول می کشد، در حالی که نیازمندی، معماری و راستی آزمایی سرسختانه با سرعت انسان پیش می روند. نتیجه، نسخه سریع تر چرخه قدیمی نیست؛ گردش کاری متفاوت است که در آن مرز فازها محو می شود، تکرارها از هفته به دقیقه می رسد و توسعه دهنده از پیاده ساز اصلی به طراح سامانه و داور کیفیت بدل می شود.

درباره سرعت تغییر

تصویر فاز به فاز زیر، وضعیت میانه ۲۰۲۶ است و به سرعت در حال جابه جایی است. نشانه ها می گویند فشرده سازی از پیاده سازی فراتر می رود: تیم ها همین حالا گردش کارهایی را می آزمایند که توسعه دهنده مستقیم از مشخصات به بازبینی می رود و عامل ها پیاده سازی، آزمون و استقرار را در پس زمینه انجام می دهند. آنچه ثابت می ماند، قضاوت انسانی، ذائقه و مهارت راستی آزمایی است.

تحول هر فاز

نیازمندی و برنامه ریزی

نیازمندی، تاریخاً فازی بوده که شکاف میان قصد و پیاده سازی در آن بیشترین عرض را داشته است. ابزارهای امروزی مستقیماً در پالایش نیازمندی مشارکت می کنند: تولید داستان کاربر از خلاصه محصول، شناسایی حالت های مرزی ای که انسان از قلم می اندازد، ساخت طرحواره API از توصیف زبان طبیعی، و ساخت نمونه اولیه تعاملی از سند مشخصات. **نیازمندی دیگر سندی نیست که میان تیم ها دست به دست شود؛ گفت و گویی است که هم زمان مشخصات و پیاده سازی اولیه تولید می کند.**

طراحی و معماری

معماری سرسخت ترین فاز در برابر خودکارسازی است، و دلیل خوبی هم دارد: تصمیم های معماری بنیاداً درباره موازنه اند – سازگاری در برابر دسترس پذیری، پیچیدگی در برابر انعطاف، ساخت در برابر خرید – و این موازنه ها به زمینه کسب و کار، قیدهای سازمانی و ملاحظات راهبردی بلندمدت وابسته اند. هوش مصنوعی در **اجرای** تصمیم

معماری عالی است: با یک سند معماری روشن، می‌تواند کل اسکلت برنامه را بسازد، الگوها را در ماژول‌ها یکدست کند و کد تازه را با قراردادهای موجود هماهنگ نگه دارد.

پیاده‌سازی

عامل‌های امروزی می‌توانند یک ویژگی کامل را از توصیف زبان طبیعی بسازند و تغییرهای چندفایلی سازگار تولید کنند. سود بهره‌وری واقعی است: پیمایش‌های صنعتی ۲۵ تا ۳۹ درصد بهبود گزارش می‌کنند.

تصویر پیچیده‌تر از تیتراها

پژوهش METR نشان داد توسعه‌دهندگان باتجربه در برخی کارها با دستیار هوش مصنوعی ۱۹٪ **کندتر** شدند؛ عمدتاً به سبب زمانی که صرف راستی‌آزمایی، اشکال‌زدایی و اصلاح خروجی می‌شد. هوش مصنوعی کار پیاده‌سازی را حذف نمی‌کند؛ آن را از نوشتن به بازبینی، هدایت و راستی‌آزمایی تبدیل می‌کند.

آزمون و تضمین کیفیت

آزمون کد تولیدشده با هوش مصنوعی یعنی سنجش نه فقط آنچه عامل تولید کرد، بلکه **مسیری که طی کرد**. ارزیابی خروجی، مصنوع نهایی را می‌سنجد (کامپایل می‌شود؟ آزمون‌ها سبزند؟) و ارزیابی مسیر، کل توالی فراخوان ابزارها و استدلال میانی را. هر دو لازم‌اند، چون خروجی روانی که گام‌های راستی‌آزمایی‌اش را میان‌بر زده، شکست خطرناک‌تری از خروجی با خطای آشکار است.

هوش مصنوعی خود تولید آزمون را هم دگرگون می‌کند: عامل‌ها می‌توانند حالت‌های مرزی و آزمون‌های مبتنی بر ویژگی تولید کنند که به ذهن انسان نمی‌رسد. مهم‌تر از آن، آزمون‌ها و ارزیابی‌ها به سازوکار اصلی **انتقال قصد** به عامل بدل می‌شوند: یک مجموعه ارزیابی خوب‌نوشته، دقیق‌تر از هر درخواست زبان طبیعی می‌گوید «درست» یعنی چه.

چرخ‌طیار کیفیت پیوسته

ارزیابی در برابر مجموعه محک ← خوشه‌بندی علل شکست ← بهینه‌سازی درخواست‌ها یا ابزارهایی که باعثش بودند ← راستی‌آزمایی اصلاح در برابر مجموعه بازگشتی ← پایش ترافیک تولید برای یافتن حالت‌های شکست تازه. هر دور، دور بعدی را تقویت می‌کند.

بازبینی کد و استقرار

هوش مصنوعی به‌عنوان بازبین نوبت اول عمل می‌کند: یافتن اشکال‌های محتمل، نقض سبک، آسیب‌پذیری‌های امنیتی و مسائل کارایی پیش از آنکه بازبین انسانی کد را ببیند. این جایگزین بازبینی انسانی نیست – تصمیم‌های وابسته به زمینه درباره طراحی، نگهداشت‌پذیری و هم‌راستایی راهبردی هنوز قضاوت انسانی می‌خواهند – اما بار

شناختی بازیبن را به شکل معناداری کم می‌کند. خطوط لوله استقرار هم آگاه‌تر می‌شوند: پایش سلامت انتشار، بازگرداندن خودکار انتشار مشکل‌دار و پیش‌بینی ریسک بر پایه ماهیت و دامنه تغییر.

نگهداری و تکامل

شاید کم‌تخمین‌زده‌ترین تحول همین‌جا باشد. کدپایه‌های قدیمی‌ای که برای تازه‌واردان نفوذناپذیر بودند، اکنون قابل پیمایش، فهم و تغییرند. این برای بدهی فنی پیامد بزرگی دارد: کدی که «دست‌زدن به آن پرریسک» بود، چون فقط نویسندگان اصلی‌اش می‌فهمیدندش، حالا می‌تواند ایمن‌سازی و نوسازی شود. مهاجرت میان چارچوب‌ها، به‌روزرسانی API‌های منسوخ و نوسازی مجموعه‌آزمون‌ها – کارهایی که پیش‌تر آن‌قدر خسته‌کننده و پرریسک بودند که عملاً هرگز انجام نمی‌شدند.

مدل کارخانه

مدل ذهنی‌ای که همه این تحولات را به هم گره می‌زند، **مدل کارخانه** است. در این مدل، خروجی اصلی توسعه‌دهنده کد نیست؛ **سامانه‌ای است که کد تولید می‌کند**. این سامانه شامل مشخصات و زمینه‌ای است که تعریف می‌کند چه باید ساخته شود؛ عامل‌هایی که مشخصات را به پیاده‌سازی ترجمه می‌کنند؛ آزمون‌ها و دروازه‌های کیفیت که درستی را می‌سنجند؛ حلقه‌های بازخوردی که شکست‌ها را برای اصلاح به عامل برمی‌گردانند؛ و حفاظ‌هایی که عامل را به رفتار ایمن و قابل‌پیش‌بینی محدود می‌کنند.

مدیر کارخانه هر قطعه را با دست نمی‌سازد. او خط تولید را طراحی می‌کند و کنترل کیفیت را تضمین می‌کند. توسعه‌دهنده امروز، سامانه توسعه را طراحی می‌کند و مطمئن می‌شود خروجی‌اش استاندارد لازم را برآورده می‌کند.

۶. مهندسی مهار: آنچه مدل را احاطه می‌کند

وقتی سازندگان تازه با عامل‌ها کار می‌کنند، وسوسه‌ای هست که مدل را با کل سامانه یکی بگیرند: مدل تازه‌ای می‌آید و عامل باهوش‌تر می‌شود؛ مدل قدیمی‌تر و عامل ضعیف‌تر. مدل توضیح همه‌چیز می‌شود، خوب و بد. این شهود غلط است و به سرمایه‌گذاری‌های اشتباه می‌انجامد.

معادله مفید

عامل = مدل + مهار. مدل خام، عامل نیست. وقتی عامل می‌شود که مهار به آن وضعیت، اجرای ابزار، حلقه بازخورد و قیدهای قابل‌اعمال بدهد. رفتاری که هنگام کار با Claude Code، Cursor، Codex، Antigravity، Aider یا Cline تجربه می‌کنید، بیش از آنکه محصول مدل زیرین باشد، محصول کاری است که مهار انجام می‌دهد.

داخل مهار چیست؟

- **دستورها و فایل‌های قاعده:** متنی که می‌گوید عامل کیست، به چه اهمیت می‌دهد و از چه کاری منع شده است. شامل `GEMINI.md` ، `CLAUDE.md` ، `AGENTS.md` ، فایل‌های مهارت و درخواست‌های زیرعامل‌ها.
 - **ابزارها:** توابع، سرورهای MCP و API‌هایی که عامل می‌تواند صدا بزند – به‌علاوهٔ نثری که به مدل می‌گوید کی و چگونه صدایشان بزند.
 - **جعبه‌شن و محیط‌های اجرا:** کد عامل واقعاً کجا اجرا می‌شود، به چه چیزی دسترسی دارد و به چه چیزی نمی‌تواند برسد.
 - **منطق ارکستراسیون:** زایش زیرعامل، مسیریابی مدل، تحویل کار بین متخصص‌ها و قواعدی که تعیین می‌کنند هرکدام کی فعال شوند.
 - **حفاظ‌ها یا قلاب‌ها:** کد قطعی که در نقاط مشخصی از چرخهٔ عمر اجرا می‌شود: پیش از فراخوان ابزار، پس از ویرایش فایل، پیش از کامیت. قلاب‌ها جای چیزهایی‌اند که عامل هرگز نباید فراموش کند اما اغلب می‌کند.
 - **رصدپذیری:** گزارش‌ها، ردیابی‌ها، ارزیابی‌ها، سنجش هزینه و تأخیر. بدون رصدپذیری، هیچ راهی نیست بفهمیم عامل خوب کار می‌کند یا بی‌سروصدا منحرف شده است.
- اگر این سطح گسترده به‌نظر زیاد می‌آید، واقعاً هم زیاد است. و مهم‌تر: این سطح، سطح تیم شماست، نه فروشندهٔ مدل.

مهار در فازهای چرخه عمر

فاز	اجزای فعال مهار	کاری که انجام می‌شود
۱. نیازمندی، برنامه و معماری پیکربندی مهار	دستورها و فایل‌های قاعده	پیش از آنکه هوش مصنوعی کد تولیدی بنویسد، توسعه‌دهنده محیط عامل را برپا می‌کند: ساختن AGENTS.md ، تعریف قیدهای معماری، مشخص کردن ابزارهای در دسترس و قواعدی که عامل حق شکستن‌شان را ندارد.
۲. پیاده‌سازی اجرای مهار	جعبه‌شن، محیط اجرا، ابزارها	مدل کد تولید می‌کند و آن را درون جعبه‌شن ایزوله مهار اجرا می‌کند. خواندن فایل یا جست‌وجوی وب فقط از راه ابزارهایی ممکن است که مهار فراهم کرده.
۳. آزمون و تضمین کیفیت حلقه بازخورد	منطق ارکستراسیون، حفاظ‌ها	مهار محیط اجرای آزمون را فراهم می‌کند؛ اگر آزمونی بشکند، منطق ارکستراسیون خروجی خطا را می‌گیرد و به مدل برمی‌گرداند و می‌گوید دوباره تلاش کن. همین، حلقه خودکار «بیندیش ← کنش کن ← مشاهده کن» را می‌سازد.
۴. بازبینی، استقرار و نگهداری رصد مهار	قلاّب‌ها، رصدپذیری	قلاّب قطعی جلوی کامیتی را می‌گیرد که گذرواژه سخت‌کدشده دارد. لایه رصدپذیری هزینه توکن، تأخیر و انحراف عامل را دنبال می‌کند تا مهندسان بتوانند ممیزی کنند که عامل چرا تصمیم خاصی گرفت.

اثر مهار قابل اندازه‌گیری است

محک‌های عمومی اندازه این اثر را ملموس می‌کنند. در Terminal Bench 2.0 یک تیم فقط با تغییر مهار – بدون هیچ تغییری در مدل – عاملی را از بیرون سی‌تای برتر به پنج‌تای برتر رساند. پژوهشی جداگانه در LangChain با دست‌کاری تنها درخواست سامانه، ابزارها و میان‌افزار پیرامون مدلی ثابت، امتیاز همان محک را ۱۳/۷ واحد بالا برد.

گذار از کدنویسی شهودی به مهندسی عامل محور صرفاً دربارهٔ ابزاری که به کار می‌برید نیست – یک توسعه‌دهنده می‌تواند با عامل دقیقاً یکسان، هم شهودی کد بزند و هم مهندسی عامل محور کند. آنچه تعیین می‌کند، این است که چقدر آگاهانه مهار را پیکربندی و اعمال می‌کنید.

نسخهٔ روزمرهٔ این مشاهده

وقتی عامل کار اشتباهی می‌کند، اولین گزینه سرزنش مدل است. اما اغلب رد خطا به یک ابزار غایب، یک قاعدهٔ مبهم، یک حفاظ نبوده یا پنجرهٔ زمینه‌ای انباشته از نویز می‌رسد. بیشتر شکست‌های عامل، اگر صادقانه بررسی شوند، شکست پیکربندی‌اند.

۷. نقش در حال تحول توسعه‌دهنده

دو حالت کاری که توسعه‌دهندگان روان بین آن‌ها جابه‌جا می‌شوند:

هماهنگ‌کننده

واگذاری ناهم‌زمان: هدف‌ها را تعریف می‌کنی، به عامل‌ها می‌سپاری و نتیجه را بازبینی می‌کنی. عامل‌ها در پس‌زمینه و موازی کار می‌کنند.

رهبر ارکستر

هدایت هم‌زمان و دستی: در ویرایشگر، هم‌زمان با ظاهرشدن کد، با درخواست و اصلاح هدایتش می‌کنی و کنترل ریزدانه روی هر تغییر داری.

حالت رهبر ارکستر

این حالت برای منطق پیچیده، اشکال‌زدایی دشوار یا کار در کدپایهٔ ناآشنا مناسب است؛ جایی که باید هر تغییر را همان لحظه بفهمی. ابزارهایی مثل Cursor، Gemini Code Assist، GitHub Copilot و Windsurf عمدتاً همین حالت را پشتیبانی می‌کنند. این حالت برای مهندسان با پیشینهٔ سنتی طبیعی است و حس فهم و کنترل را حفظ می‌کند. ریسکش این است که خود توسعه‌دهنده به گلوگاه بدل شود: اگر هر فشارِ کلید را شخصاً هدایت کنی، بهرهٔ سرعت محدود می‌ماند.

حالت هماهنگ‌کننده

در این حالت، توسعه‌دهنده در سطح انتزاع بالاتری کار می‌کند و کد را خط‌به‌خط تماشا نمی‌کند. مناسب کارهای خوش‌تعریف است: رفع اشکال، پیاده‌سازی ویژگی روی الگوهای جاافتاده، مهاجرت کدپایه و تولید آزمون. ابزارهایی مثل Jules، حالت عاملی Copilot، عامل‌های پس‌زمینهٔ Cursor و Claude Code این حالت را با اجرای ناهم‌زمان کار، اغلب در محیط‌های جعبه‌شنی با دسترسی کامل به مخزن و ابزار ساخت، پشتیبانی می‌کنند.

این حالت مجموعه مهارت متفاوتی می‌خواهد؛ به‌جای تسلط عمیق بر نحو و اصطلاحات زبان:

- **مشخصه‌نویسی:** تعریف کار آن‌قدر دقیق که عامل بدون ابهام اجرایش کند.
- **تجزیه:** شکستن کار بزرگ به واحدهایی با اندازه مناسب برای اجرای عامل.
- **ارزیابی:** سنجش سریع اینکه خروجی عامل استاندارد کیفی را برآورده می‌کند یا نه.
- **طراحی سامانه:** طراحی قیدها، آزمون‌ها و حلقه‌های بازخوردی که عامل‌ها را بارور نگه می‌دارد.

مسئله ۸۰ درصد

چالشی پایدار در توسعه کمک‌گرفته از هوش مصنوعی: عامل‌ها می‌توانند حدود ۸۰٪ کد یک ویژگی را به‌سرعت تولید کنند، اما ۲۰٪ باقی‌مانده – حالت‌های مرزی، مدیریت خطا، نقاط یکپارچگی و الزام‌های ظریف درستی – دانش زمینه‌ای عمیقی می‌طلبد که مدل‌های امروز اغلب فاقد آن‌اند.

ماهیت خطاها هم تغییر کرده است: از اشتباه‌های نحوی ساده به شکست‌های مفهومیِ موزیانه – فرض‌های غلط دربارهٔ منطق کسب‌وکار، نپرسیدن دربارهٔ نیازمندی مبهم، جاافتادن حالت مرزی، و تصمیم‌های معماری‌ای که بار نگهداری بلندمدت می‌سازند. این خطاها دقیقاً به این دلیل سخت‌تر دیده می‌شوند که کد «درست به‌نظر می‌رسد» و حتی ممکن است از آزمون‌های پایه رد شود.

وضعیتی که کارآمدترین توسعه‌دهندگان می‌گیرند

هوش مصنوعی را برای آنچه در آن خوب است به کار می‌گیرند (پیاده‌سازی سریع کارهای خوش‌تعریف) و توجه خودشان را برای آنچه هوش مصنوعی در آن ضعیف است نگه می‌دارند (نیازمندی مبهم، موازنه معماری، راستی‌آزمایی درستی). آن‌ها نمی‌کوشند با پذیرفتن همه‌چیز سریع‌تر شوند؛ می‌کوشند با تمرکز تخصص‌شان بر جایی که بیشترین اهمیت را دارد سریع‌تر شوند.

۸. عامل‌های کدنویسی در عمل

سه جایگاه در روز کاری

در ترمینال

عاملی که از خط فرمان اجرا می‌شود، هدف را به زبان ساده می‌گیرد و در کل کدپایه کار می‌کند: دسترسی کامل به سامانه فایل، ویرایش چندفایلی، اجرای ابزار و آزمون، و تکرار بر پایه نتیجه. جدی‌ترین کدنویسی شهودی امروز اینجا رخ می‌دهد.

در ویرایشگر

تکمیل درون‌خطی، پنل گفت‌وگو برای توضیح یا تغییر کد در جا، و آگاهی از کل کدپایه. جایی که بیشتر افراد نخستین‌بار با هوش مصنوعی روبه‌رو می‌شوند و کار «در جریان» می‌ماند.

در پس‌زمینه

عاملی که کاری را می‌گیرد و ساعت‌ها در جعبه‌شن ابری خودگردان کار می‌کند و معمولاً یک درخواست ادغام تحویل می‌دهد. توسعه‌دهنده کار را می‌سپارد و بعداً بازبینی می‌کند.

در عمل، عامل ویرایشگر وقتی به کار می‌آید که وسط نوشتن کدید و پیشنهاد یا توضیح سریع می‌خواهید. عامل ترمینال مناسب کارهای چندفایلی، کاوش کدپایه ناآشنا و کارهایی است که عامل باید کد اجرا کند و به مشاهده‌اش واکنش نشان دهد. عامل پس‌زمینه مناسب کاری است که بتوانید در یک پاراگراف توصیفش کنید و بروید: رفع اشکالی شناخته‌شده، تولید مجموعه آزمون، یا مهاجرت کد از یک چارچوب به چارچوب دیگر. **نقطه شروع درست به ماهیت کار بستگی دارد، نه به اینکه کدام دسته روی نردبان خودگردانی بالاتر می‌ایستد.**

وقتی خود محصول یک عامل است

تا اینجا سخن از به‌کارگیری عامل‌ها برای ساختن نرم‌افزار بود. اما اگر چیزی که باید بسازید خودش یک عامل باشد چه؟ رباتی که درخواست‌های بازپرداخت را رسیدگی می‌کند؛ دستیار پژوهشی که منابع را تطبیق می‌دهد و گزارش مستند تولید می‌کند؛ ابزار داخلی که انطباق را پایش و ناهنجاری را علامت‌گذاری می‌کند. این‌ها محصول‌اند و به ابزار، حافظه، ارزیابی و زیرساخت استقرار خودشان نیاز دارند.

نکته تازه این است که همان گردش کار ترمینالی که اسکریپت نمونه تولید می‌کرد، حالا به این عامل‌های تولیدی هم می‌رسد. برای نمونه، Agents CLI گوگل ابزار کوچکی است که مجموعه‌ای از مهارت‌ها را به عامل کدنویسی دلخواه شما اضافه می‌کند و کل چرخه ساخت، ارزیابی، استقرار و رصد را پوشش می‌دهد:

```
# one-time setup
uvx google-agents-cli setup

# then, inside your coding agent:
> Build a support agent that answers questions from our docs.
> Evaluate it on the FAQ dataset.
> Deploy it to Agent Engine.
```

پشت همین چند جمله، عامل کدنویسی پروژه را از قالب می‌سازد، کد را می‌نویسد، مجموعه ارزیابی تولید می‌کند، آن را اجرا می‌کند، روی زمان اجرای عامل مستقر می‌کند و گزارش می‌دهد. توسعه‌دهنده SDK تازه‌ای یاد نمی‌گیرد؛ توصیف می‌کند و عامل از مهارت‌ها استفاده می‌کند.

هماهنگی میان چند عامل

برای حالت‌های ساده، وضعیت مشترک نشست کافی است؛ برای دسترسی به ابزار، MCP؛ و برای واگذاری کار میان عامل‌ها، پروتکل A2A. تیم مهندسی Anthropic در اوایل ۲۰۲۶ آزمایشی منتشر کرد که در آن تیم‌هایی از عامل‌ها روی چنین معماری‌ای، ظرف دو هفته یک کامپایلر زبان C را به زبان Rust ساختند؛ انسان‌ها جهت می‌دادند و خروجی را بازبینی می‌کردند اما پیاده‌سازی را نمی‌نوشتند. گلوگاه از نوشتن کد به مشخص کردن اینکه چه باید بشود و راستی‌آزمایی اینکه شده منتقل شد.

۹. اقتصاد توسعه مبتنی بر هوش مصنوعی

گفت‌وگو درباره اثر هوش مصنوعی معمولاً با «سرعت توسعه‌دهنده» آغاز و تمام می‌شود. اما برای رهبران فنی، سنجه مهم‌تر هزینه کل مالکیت است. برای فهم هزینه واقعی باید دید هر گردش کار، بار مالی را چگونه میان CapEx (سرمایه‌گذاری اولیه برای ساختن) و OpEx (هزینه جاری اجرا، رفع و نگهداری) جابه‌جا می‌کند. و در عصر هوش مصنوعی، OpEx را عمده‌تاً اقتصاد توکن تعیین می‌کند.

بدهی پنهان کدنویسی شهودی: سرمایه‌گذاری کم، هزینه عملیاتی زیاد

در نگاه اول، کدنویسی شهودی فوق‌العاده مقرون‌به‌صرفه است: مانع ورود تقریباً صفر است — یک اشتراک ماهانه و چند درخواست ساده. اما اقتصادش بار عملیاتی انباشته‌ای را پنهان می‌کند:

- **نرخ سوخت توکن:** ریختن فایل‌های حجیم و بی‌ساختار در پنجرهٔ زمینه و درخواست مکرر برای اصلاح اشتباهات راستی‌آزمایی‌نشدهٔ خودِ مدل، حلقه‌ای گران با نرخ موفقیت پایین در نوبت اول می‌سازد.
- **مالیات نگه‌داری:** کدی که با درخواست‌های سرِدستی نوشته شده، اغلب انسجام ساختاری ندارد. شش ماه بعد که اشکالی پیدا شود، مهندسان انسانی باید روزها صرف مهندسی معکوس کد «اسپاگتی» تولیدشده کنند.
- **جبران امنیتی:** بدون مهار ارزیابی خودکار، تولید سریع کد یعنی تولید سریع آسیب‌پذیری. هزینهٔ رفع نقص امنیتی در تولید، به‌شکل نمایی بیشتر از گرفتن همان نقص در فاز طراحی است.

سرمایه‌گذاری مهندسی عامل‌محور: سرمایه‌گذاری زیاد، هزینهٔ عملیاتی کم

مهندسی عامل‌محور این مدل اقتصادی را وارونه می‌کند. پیش از تولید نخستین خط کد تولیدی، سرمایه‌گذاری آگاهانه‌ای از زمان مهندسی لازم است: طراحی طرح‌واره‌های API، ساخت مجموعه‌آزمون‌های قطعی و — از همه مهم‌تر — ساختاردهی زمینهٔ عامل. این هزینهٔ اولیه بیشتر است، اما هزینهٔ نهایی عرضه و نگه‌داری هر ویژگی به‌شدت افت می‌کند، چون عامل درون «کارخانه‌ای» به‌دقت حکمرانی‌شده کار می‌کند و خروجی‌اش از نظر ساختاری سالم، از پیش آزمون‌شده و هم‌راستا با استانداردهای سازمان است.

مهندسی زمینه به‌عنوان اهرم مالی

در اقتصاد توکن، مهندسی زمینه فقط یک مهارت فنی نیست؛ یک راهبرد مالی است. مدل‌ها بابت هر تکه اطلاعاتی که می‌فرستید هزینه می‌گیرند و فرستادن مخزنی ۱۰۰,۰۰۰ توکنی در هر درخواست، در مقیاس، اقتصادی نیست. مهندسی زمینه مؤثر تضمین می‌کند که مدل بارِ متراکم و پرسپیکال می‌گیرد (مثل یک AGENTS.md دقیق و حفاظ‌های معماری) نه انبوهی پرنویز. با فراهم‌کردن زمینهٔ درست از همان ابتدا، نرخ موفقیت نوبت اول بالا می‌رود و حلقه‌های پرهزینهٔ آزمون‌وخطا حذف می‌شوند.

مسیریابی هوشمند مدل

در گردش‌کار شهودی، معمولاً یک مدل پیشروی عظیم برای هر تعاملی به کار می‌رود — یعنی پرداخت قیمت درجه‌یک حتی برای اصلاح یک غلط تایپی یا نوشتن یک آزمون واحد ساده. مدل کارخانهٔ خوب‌طراحی‌شده از این اتلاف پرهیز می‌کند: مدل‌های بزرگ برای کارهای بسیار پیچیده (نیازمندی، معماری، پیاده‌سازی اولیه) و مسیریابی خودکار کارهای قطعی‌تر و کم‌پیچیدگی‌تر (تولید آزمون، بازبینی کد، پایش CI/CD) به مدل‌های کوچک‌تر، سریع‌تر و بسیار ارزان‌تر.

۱۰. از کجا شروع کنیم

اصل راهنما

هوش مصنوعی فرهنگ مهندسی‌ای را که در آن فرود می‌آید تقویت می‌کند. توصیه‌های زیر، همین اصل را به عمل ترجمه می‌کنند.

برای توسعه‌دهنده منفرد

1. یک `AGENTS.md` (یا معادلش) برای پروژه بسازید. قراردادی را انتخاب کنید که با عامل مورد استفاده‌تان هم‌خوان است. با ده خط شروع کنید: پشتۀ فناوری، قراردادهای، قواعد سخت، گردش کار. هر بار عامل کاری کرد که نباید تکرار شود، یک قاعده اضافه کنید.
2. مجموعه‌ای از مهارت‌ها را برای عامل کدنویسی‌تان نصب کنید تا ساخت، ارزیابی، استقرار و بهینه‌سازی عامل‌ها را پوشش دهند.
3. یک گردش کار تکراری را انتخاب و به نخستین عامل‌تان تبدیل کنید. ساخت یک عامل از ابتدا تا انتها بیش از خواندن دربارهٔ صد عامل می‌آموزد.
4. آزمون‌ها و ارزیابی‌ها را پیش از تولید کد بنویسید. این دو با هم قرارداد شما با هوش مصنوعی‌اند و قصد را دقیق‌تر از هر درخواست زبان طبیعی منتقل می‌کنند.
5. هر خطی را که قرار است منتشر شود بازبینی کنید. به هر چیزی که «زیرکانه» به نظر می‌رسد مشکوک باشید. بررسی کنید بسته‌های واردشده واقعاً وجود دارند. کدی که تیم نمی‌فهمدش، به هزینه‌اشکال‌زدایی‌ای بدل می‌شود که تیم توان پرداختش را ندارد.
6. مهارت‌های بنیادی خود را زنده نگه دارید. هوش مصنوعی کار روتین را می‌گیرد تا شما روی کار دشوار تمرکز کنید؛ اما این ترتیب فقط وقتی کار می‌کند که اشکال‌زدایی، طراحی سامانه و شهود کارایی و درستی شما تیز بماند.

برای رهبران مهندسی

1. مهندسی زمینه را به یک رویۀ مهندسی درجه یک تبدیل کنید. با `AGENTS.md`، درخواست‌های سامانه، مجموعه‌های ارزیابی و کتابخانه‌های مهارت مثل کد رفتار کنید: بازبینی در درخواست ادغام، نسخه‌بندی با پروژه و مالکیت مشخص. بدون این انضباط، مهار منحرف می‌شود و رفتار عامل در تیم تکرارناپذیر می‌گردد.
2. معیار را روی ارزیابی بگذارید، نه نمایش. یک نمایش موفق ثابت می‌کند عامل «یک‌بار» می‌تواند؛ مجموعه ارزیابی قبول‌شده ثابت می‌کند «قابل‌اتکا» می‌تواند. اما ارزیابی بدون سنجۀ روشن هیچ چیزی را نمی‌سنجد:

مشخص کنید چه چیزی را نمره می‌دهید – موفقیت کار، کیفیت استفاده از ابزار، انطباق مسیر، توهم، و کیفیت پاسخ.

3. **بازبینی کد را برای کد تولیدشده بازطراحی کنید.** کد تولیدشده به همان اندازه یا بیشتر موشکافی می‌خواهد، با توجه ویژه به وابستگی‌های توهمی، مدیریت خطای ناکافی و شکاف‌های ظریف درستی که در نگاه اول درست به نظر می‌رسند.
4. **مرز کار نمونه‌سازی و کار تولیدی را در هنجارهای تیم صریح کنید.** کدنویسی شهودی سرعتِ درستِ کاوش است و مهندسی عامل‌محور انضباطِ درستِ تولید. تیم‌هایی که این مرز را مبهم می‌گذارند، نمونه‌های اولیه‌ای تولید می‌کنند که تصادفاً منتشر می‌شوند.
5. **روی اجزای مهار به‌عنوان دارایی مشترک تیم سرمایه‌گذاری کنید.** درخواست‌های سامانه قابل استفاده مجدد، کتابخانه‌های مهارت، اتصال‌های سرور MCP و مهارهای ارزیابی، ارزش‌شان در پروژه‌ها انباشته می‌شود. با آن‌ها مثل زیرساخت رفتار کنید.

برای سازمان‌ها

1. **توسعه کمک‌گرفته از هوش مصنوعی را سرمایه‌گذاری مهندسی بدانید، نه یک قابلیت بهره‌وری.** عرضه عامل کدنویسی بدون داربست ارزیابی، رصدپذیری و استاندارد معماری، سرعت بدون کیفیت تولید می‌کند و بدهی فنی را سریع‌تر از توان بازپرداخت تیم انباشته می‌سازد.
2. **پیش از مقیاس‌گرفتن، بستر تولید را بسازید.** ارزیابی مسیر و پاسخ نهایی در CI، ردیابی هر اجرای عامل، مجوزهای محدودشده به ازای هر عامل، و بازبینی امنیتی تنظیم‌شده برای حالت‌های شکست کد تولیدشده – این بستر باید پیش از انتشار نخستین عامل تولیدی آماده باشد، نه بعد از آن.
3. **استانداردهای باز را برای ابزار و ارتباط میان‌عاملی بپذیرید.** MCP برای دسترسی به ابزار و A2A برای واگذاری میان عامل‌ها در حال تبدیل‌شدن به بافت پیوندی سامانه‌های چندعاملی‌اند.
4. **برای تیم‌های ترکیبی انسان و عامل برنامه‌ریزی کنید.** قوی‌ترین نتایج تولیدی از معماری‌هایی آمده که انسان جهت می‌دهد، عامل پیاده‌سازی می‌کند و پروتکل‌های تحویل روشن، مرز را اداره می‌کنند.
5. **استخدام و توسعه مهارت را حول قضاوت بازتعریف کنید.** با خودکارت‌ر شدن پیاده‌سازی، گلوگاه به مشخصه‌نویسی، ارزیابی، قضاوت معماری و بازبینی منتقل می‌شود.

۱۱. جمع‌بندی و سه اصل ماندگار

گذار از نحو به قصد پیش‌بینی آینده نیست؛ واقعیت امروز است. توسعه‌دهندگان همین حالا زمان بیشتری صرف توصیف «چه می‌خواهند» می‌کنند تا «چگونه ساخته شود». پرسش این نیست که آیا این دگرگونی رخ می‌دهد، بلکه این است که افراد، تیم‌ها و سازمان‌ها چقدر مؤثر از آن عبور می‌کنند.

۱. ساختار مقیاس می‌گیرد، شهود نه

کدنویسی شهودی برای کاوش، نمونه‌سازی و پروژه شخصی رویکردی معتبر است. اما برای نرم‌افزاری که سازمان به آن تکیه می‌کند، انضباط مهندسی عامل محور اختیاری نیست. فاصله «به‌نظر کار می‌کند» تا «در همه شرایط درست کار می‌کند»، همان جایی است که قطعی سرویس، آسیب‌پذیری امنیتی و کابوس نگه‌داری زندگی می‌کنند.

۲. هوش مصنوعی فرهنگ مهندسی را

تقویت می‌کند

سازمان‌هایی که رویه‌های آزمون قوی، استانداردهای معماری روشن و بازبینی کد سالم دارند، به‌مراتب ارزش بیشتری از توسعه کمک‌گرفته از هوش مصنوعی می‌گیرند. هوش مصنوعی ضریب نیرو است و هم قوت‌ها و هم ضعف‌ها را بزرگ می‌کند.

۳. نقش انسان در حال تحول است، نه

کوچک‌شدن

سازندگان که معماری را می‌فهمند، می‌توانند مشخصات دقیق تعریف کنند، خروجی را نقدانه بسنجند و سامانه‌های مؤثری از قید و بازخورد طراحی کنند، از همیشه ارزشمندترند.

تولید حل شده است. راستی‌آزمایی، قضاوت و جهت‌دهی، پیشه تازه‌اند.

۱۲. خودآزمایی

پیش از جلسه بعد، پاسخ این پرسش‌ها را برای خودتان بنویسید. اگر برای یکی پاسخ روشنی ندارید، به بخش مربوط برگردید.

۱. تفاوت ربات گفت‌وگو و عامل را در یک جمله بیان کنید. کدام جزء از پنج جزء، این تفاوت را می‌سازد؟

۲. چرا «استفاده از هوش مصنوعی» معیار خوبی برای تشخیص کدنویسی شهودی از مهندسی عامل محور نیست؟ معیار درست چیست؟

۳. یک مثال بزنید که در آن آزمون کافی است و ارزیابی لازم نیست؛ و یک مثال که عکس آن صادق است.

۴. سه چیز را نام ببرید که در پروژه فعلی شما باید در زمینه ایستا باشند و سه چیز که باید پویا بمانند. چرا؟

۵. معادله «عامل = مدل + مهار» چه پیامد عملی‌ای برای بودجه تیم شما دارد؟

۶. آخرین باری که عامل اشتباهی کرد، کدام جزء مهار غایب بود؟

7. در کدام کارها شما «رهبر ارکستر» هستید و در کدامها «هماهنگ‌کننده»؟ آیا این انتخاب آگاهانه است؟

8. در مسئله ۸۰ درصد، ۲۰٪ باقی‌مانده آخرین ویژگی‌ای که ساختید چه بود؟

9. اگر بخواهید هزینه توکن تیم را نصف کنید بی‌آنکه کیفیت افت کند، از کجا شروع می‌کنید؟

10. یک قاعده سخت بنویسید که امروز باید در AGENTS.md پروژه‌تان باشد اما نیست.

منبع: این جزوه خلاصه و بازنویسی آموزشی مقاله The New SDLC With Vibe Coding: From ad-hoc prompting to Agentic Engineering نوشته آدی عثمانی، شوبهام سابو و سوکراتیس کارتاکیس است (مجموعه مقالات عامل‌ها، می ۲۰۲۶). ترجمه کامل فارسی مقاله در همین پوشه در دسترس است.

دوره مهندسی عامل‌محور و کدنویسی شهودی – جزوه جلسه اول